



# Rule of 0,1,2,5,6,7,8,9,10?

JASON TURNER



# Rules of 0,1,2,3,4,5,6,7,8,9,10?



# Jason Turner

## C++ Weekly

- Weekly videos since March, 2016
- 125k+ subscribers, 520+ weeks straight

<https://www.youtube.com/@cppweekly>



# Jason Turner

- Author
  - C++ Best Practices, C++23 Best Practices
  - OpCode, Copy and Reference, Object Lifetime Puzzlers
  - Programming Puzzles Big Book
  - <https://amzn.to/3xWh8Ox>
  - [https://leanpub.com/u/jason\\_turner](https://leanpub.com/u/jason_turner)



# Jason Turner

- Developer
  - <https://cppbestpractices.com>
  - <https://github.com/lefticus>
  - <https://github.com/cpp-best-practices>
- Microsoft MVP for C++, 2015-present



# Jason Turner - Training

<https://emptycrate.com/training.html>

How to get my training:

1. Have me come to your company on-site for dynamic customized training where you already are - generally the most economical option for groups (CA, DE, NL, RO, CZ, JP, US, PL, SE, ...)
2. Come to a conference workshop
  - ACCU On Sea (Folkestone, UK, ~June)
  - CppCon (Aurora, CO, US, ~Sept)
  - NDC TechTown (Kongsberg, NO, ~Sept)
  - C++ Under the Sea (Breda, NL, ~Oct)



And possibly others

# My Motivations



Copyright Jason Turner

@lefticus

[emptycrate.com/go.html](http://emptycrate.com/go.html)

# My Motivations

- The world runs on software. Basically everything does.



# My Motivations

- The world runs on software. Basically everything does.
- Have you ever had to reboot your toaster? (I have!)



# My Motivations

- The world runs on software. Basically everything does.
- Have you ever had to reboot your toaster? (I have!)
- I want to live in a world with better, safer software, cars, spaceships, toasters...



# My Motivations

- The world runs on software. Basically everything does.
- Have you ever had to reboot your toaster? (I have!)
- I want to live in a world with better, safer software, cars, spaceships, toasters...
- This is why I've given away so much material and continue to do so.



# My Motivations

- The world runs on software. Basically everything does.
- Have you ever had to reboot your toaster? (I have!)
- I want to live in a world with better, safer software, cars, spaceships, toasters...
- This is why I've given away so much material and continue to do so.
- This is also why I focus on Best Practices in my training material.



# My Motivations

- The world runs on software. Basically everything does.
- Have you ever had to reboot your toaster? (I have!)
- I want to live in a world with better, safer software, cars, spaceships, toasters...
- This is why I've given away so much material and continue to do so.
- This is also why I focus on Best Practices in my training material.
- The work you are doing matters. It affects peoples lives!



# Jason Turner

<https://emptycrate.com/go.html>



# About my Talks

- Avoid sitting in the back
- Please interrupt and ask questions, yell things out, I'll repeat it for the room
- This is approximately how my training days look, as interactive as reasonable



# The Rule of 0,1,2,3,4,5,6,7,8,9,10...



# What is This Talk About?

I thought I knew before I started researching and working on it.



# What is This Talk About?

I thought I knew before I started researching and working on it.

Seems I ended up a down a rabbit hole.



# What is This Talk About?

I thought I knew before I started researching and working on it.

Seems I ended up a down a rabbit hole.

So let's have some fun!



# What are the special member functions?



# What are the special member functions?

These are the functions the compiler will generate for you, if you do nothing at all.



# What are the special member functions?

These are the functions the compiler will generate for you, if you do nothing at all.

How many special members are there?



# What are the special member functions?

These are the functions the compiler will generate for you, if you do nothing at all.

How many special members are there?

1. Default constructor



# What are the special member functions?

These are the functions the compiler will generate for you, if you do nothing at all.

How many special members are there?

1. Default constructor
2. Copy Constructor



# What are the special member functions?

These are the functions the compiler will generate for you, if you do nothing at all.

How many special members are there?

1. Default constructor
2. Copy Constructor
3. Move Constructor



# What are the special member functions?

These are the functions the compiler will generate for you, if you do nothing at all.

How many special members are there?

1. Default constructor
2. Copy Constructor
3. Move Constructor
4. Copy Assignment Operator



# What are the special member functions?

These are the functions the compiler will generate for you, if you do nothing at all.

How many special members are there?

1. Default constructor
2. Copy Constructor
3. Move Constructor
4. Copy Assignment Operator
5. Move Assignment Operator



# What are the special member functions?

These are the functions the compiler will generate for you, if you do nothing at all.

How many special members are there?

1. Default constructor
2. Copy Constructor
3. Move Constructor
4. Copy Assignment Operator
5. Move Assignment Operator
6. Destructor



# What are the special member functions?

These are the functions the compiler will generate for you, if you do nothing at all.

How many special members are there?

1. Default constructor
2. Copy Constructor
3. Move Constructor
4. Copy Assignment Operator
5. Move Assignment Operator
6. Destructor

All 6 of the 5 special member functions.



# What is the signature of the default constructor?



# What is the signature of the default constructor?

```
1 struct Data {  
2     Data(); /// default constructor  
3 };
```



# What is the signature of the default constructor?

```
1 struct Data {  
2     Data(); /// default constructor  
3 };
```

Are there any other valid default constructor signatures?



# What is the signature of the default constructor?

```
1 struct Data {  
2     Data(); /// default constructor  
3 };
```

Are there any other valid default constructor signatures?

Sure, infinitely many!

```
1 struct Data {  
2     Data(int i = 2);  
3 };
```



# What is the signature of the default constructor?

```
1 struct Data {  
2     Data(); /// default constructor  
3 };
```

Are there any other valid default constructor signatures?

Sure, infinitely many!

```
1 struct Data {  
2     Data(int i = 2);  
3 };
```

We can constrain / add `constexpr`, `[[nodiscard]]` or `noexcept`, but it's still a constructor invocable with 0 parameters.



# What is the signature of the **copy** **constructor**?



# What is the signature of the **copy** constructor?

```
1 struct Data {  
2     Data(const Data &); /// copy constructor  
3 };
```



# What is the signature of the **copy** constructor?

```
1 struct Data {  
2     Data(const Data &); /// copy constructor  
3 };
```

Are there any other valid **copy constructor** signatures?



# Copy Constructors

```
1 struct Sub {  
2     Sub() = default;  
3     Sub(Sub &);  
4 };  
5  
6 struct Data { };  
7  
8 int main() {  
9     Data d;  
10    const Data d2{d}; /// 1. does this compile?  
11    Data d3{d2}; /// 2. does this compile?  
12 }
```

<https://godbolt.org/z/ja6YYcEnc>



# Copy Constructors

```
1 struct Sub {  
2     Sub() = default;  
3     Sub(Sub &);  
4 };  
5  
6 struct Data { };  
7  
8 int main() {  
9     Data d;  
10    const Data d2{d}; /// 1. does this compile?  
11    Data d3{d2}; /// 2. does this compile?  
12 }
```

<https://godbolt.org/z/ja6YYcEnc>

First one compiles, second one does not.



# Copy Constructors

```
1 struct Sub {  
2     Sub() = default;  
3     Sub(Sub &);  
4 };  
5  
6 struct Data { };  
7  
8 int main() {  
9     Data d;  
10    const Data d2{d}; /// 1. does this compile?  
11    Data d3{d2}; /// 2. does this compile?  
12 }
```

<https://godbolt.org/z/ja6YYcEnc>

First one compiles, second one does not.



# Copy Constructors

```
1 struct Sub {  
2     Sub() = default;  
3     Sub(Sub &);  
4 };  
5  
6 struct Data { };  
7  
8 int main() {  
9     Data d;  
10    const Data d2{d}; /// 1. does this compile?  
11    Data d3{d2}; /// 2. does this compile?  
12 }
```

<https://godbolt.org/z/ja6YYcEnc>

First one compiles, second one does not.



# Copy Constructors `[class.copy.ctor]`

What does the standard say?

*A non-template constructor for class  $X$  is a **copy constructor** if its first parameter is of type  `$x\&$` ,  `$\text{const } x\&$` ,  `$\text{volatile } x\&$`  or  `$\text{const volatile } x\&$` , and either there are no other parameters or else all other parameters have default arguments ([\[dcl.fct.default\]](#)).*



# Copy Constructors

```
1 Sub(Sub &);  
2 Sub(const Sub &);  
3 Sub(volatile Sub &);  
4 Sub(const volatile Sub &);
```

<https://godbolt.org/z/f9jeG8n9G>



# Copy Constructors

```
1 Sub(Sub &);  
2 Sub(const Sub &);  
3 Sub(volatile Sub &);  
4 Sub(const volatile Sub &);
```

<https://godbolt.org/z/f9jeG8n9G>

and also

```
1 Sub(Sub &, int i = 3);  
2 Sub(const Sub &, int i = 5);  
3 Sub(volatile Sub &, std::string s1 = "Hello", std::string s2 = "World");  
4 Sub(const volatile Sub &);
```

<https://godbolt.org/z/jav18h5cW>



# Defaulted Copy Constructors

So can I do this?

```
1 struct Data {  
2     Data(volatile Data &) = default?  
3 };
```



# Defaulted Copy Constructors

So can I do this?

```
1 struct Data {  
2     Data(volatile Data &) = default;  
3 };
```

Kind of...

```
1 <source>:6:3: warning: explicitly defaulted copy constructor is  
2 implicitly deleted because its declared type does not match the  
3 type of an implicit copy constructor [-Wdefaulted-function-deleted]  
4     6 |     Data(volatile Data &) = default;  
5       |     ^
```



# Defaulted Copy Constructors

*The implicitly-declared **copy constructor** for a class  $X$  will have the form  $x::x(const\ x\&)$  if each potentially constructed subobject of a class type  $M$  (or array thereof) has a **copy constructor** whose first parameter is of type  $const\ M\&$  or  $const\ volatile\ M\&$ . Otherwise, the implicitly-declared **copy constructor** will have the form  $x::x(x\&)$*



# Defaulted Copy Constructors

```
1 struct Sub {  
2     Sub() = default;  
3     Sub(Sub &) = default;  
4 };  
5  
6 struct Data {  
7     // what is the signature of Data's  
8     // implicitly defaulted copy constructor?  
9 };  
10  
11 int main() {  
12     Data d;  
13 }
```

<https://godbolt.org/z/E6o5MKf5q>



# Defaulted Copy Constructors

```
1 struct Sub {  
2     Sub() = default;  
3     Sub(Sub &) = default;  
4 };  
5  
6 struct Data {  
7     // what is the signature of Data's  
8     // implicitly defaulted copy constructor?  
9 };  
10  
11 int main() {  
12     Data d;  
13 }
```

<https://godbolt.org/z/E6o5MKf5q>

Data(Data &)



# Defaulted Copy Constructors

```
1 struct Sub {  
2     Sub() = default;  
3     Sub(Sub &) = default;  
4 };  
5  
6 struct Data {  
7     Data() = default;  
8     Data(const Data &) = default;  
9 };  
10  
11 int main() {  
12     Data d;  
13     auto d2 = d; /// does this compile?  
14 }
```

<https://godbolt.org/z/nh5E649zb>



# Defaulted Copy Constructors

```
1 struct Sub {  
2     Sub() = default;  
3     Sub(Sub &) = default;  
4 };  
5  
6 struct Data {  
7     Data() = default;  
8     Data(const Data &) = default;  
9 };  
10  
11 int main() {  
12     Data d;  
13     auto d2 = d; /// does this compile?  
14 }
```

<https://godbolt.org/z/nh5E649zb>

No - there's now a mismatch.



# Defaulted Copy Constructors

```
1 struct Sub {  
2     Sub() = default;  
3     Sub(Sub &) = default;  
4 };  
5  
6 struct Data {  
7     Data() = default;  
8     Data(Data &) = default;  
9     Data(const Data &) = default;  
10 };  
11  
12 int main() {  
13     Data d;  
14     auto d2 = d; /// does this compile?  
15 }
```

<https://godbolt.org/z/GrvqGzzbE>



# Defaulted Copy Constructors

```
1 struct Sub {  
2     Sub() = default;  
3     Sub(Sub &) = default;  
4 };  
5  
6 struct Data {  
7     Data() = default;  
8     Data(Data &) = default;  
9     Data(const Data &) = default;  
10 };  
11  
12 int main() {  
13     Data d;  
14     auto d2 = d; /// does this compile?  
15 }
```

<https://godbolt.org/z/GrvqGzzbE>

GCC allows it, clang does not. They both comment that one of those constructors is implicitly deleted.



# Defaulted Copy Constructors

```
1  #include <type_traits>
2
3  struct Sub {
4      Sub() = default;
5      Sub(Sub &) = default; // definitely is a copy constructor
6  };
7
8  static_assert(std::is_copy_constructible_v<Sub>); /// compiles?
9  //
```

<https://godbolt.org/z/bPGvKMYbd>



# Defaulted Copy Constructors

```
1  #include <type_traits>
2
3  struct Sub {
4      Sub() = default;
5      Sub(Sub &) = default; // definitely is a copy constructor
6  };
7
8  static_assert(std::is_copy_constructible_v<Sub>); /// compiles?
9  //
```

<https://godbolt.org/z/bPGvKMYbd>

No! `is_copy_constructible` specifically requires that it be constructible with a `const Sub &`.



# Defaulted Copy Constructors

```
1  #include <type_traits>
2
3  struct Sub {
4      Sub() = default;
5      Sub(Sub &) = default; // definitely is a copy constructor
6  };
7
8  static_assert(std::is_copy_constructible_v<Sub>); /// compiles?
9  //
```

<https://godbolt.org/z/bPGvKMYbd>

No! `is_copy_constructible` specifically requires that it be constructible with a `const Sub &`.

So is `is_copy_constructible` over specified if it narrowly requires the parameter to be `const` when the standard can implicitly create me a non-`const &` copy constructor?





Copyright Jason Turner

@lefticus

[emptycrate.com/go.html](http://emptycrate.com/go.html)

Is `is_copy_constructible`  
over specified? It requires  
the parameter to be `const`  
when the standard can  
implicitly create me a non-  
`const` & copy constructor?



# Defaulted Copy Constructors

```
1 struct Data {  
2     // all compilers allow this  
3     Data(Data &) = default;  
4     Data(const Data &) = default;  
5 };
```

<https://godbolt.org/z/GY96jezbz>





Copyright Jason Turner

@lefticus

[emptycrate.com/go.html](http://emptycrate.com/go.html)

# Thought you knew what a copy constructor was?





Copyright Jason Turner

@lefticus

[emptycrate.com/go.html](http://emptycrate.com/go.html)

# Move Constructor



# Move Constructor(s?)

What are the allowed signatures for move constructors?



# Move Constructor(s?)

What are the allowed signatures for move constructors?

*A non-template constructor for class  $X$  is a **move constructor** if its first parameter is of type  `$X\&\&$` ,  `$\text{const } X\&\&$` ,  `$\text{volatile } X\&\&$` , Or  `$\text{const volatile } X\&\&$` , and either there are no other parameters or else all other parameters have default arguments*



# Move Constructor(s?)

What are the allowed signatures for move constructors?

*A non-template constructor for class  $X$  is a **move constructor** if its first parameter is of type  `$X\&\&$` ,  `$\text{const } X\&\&$` ,  `$\text{volatile } X\&\&$` , Or  `$\text{const volatile } X\&\&$` , and either there are no other parameters or else all other parameters have default arguments*

Which of these can be defaulted, and what are the rules for their construction?



# Move Constructor(s?)

What are the allowed signatures for move constructors?

*A non-template constructor for class  $X$  is a **move constructor** if its first parameter is of type  $X\&\&$ ,  $\text{const } X\&\&$ ,  $\text{volatile } X\&\&$ , or  $\text{const volatile } X\&\&$ , and either there are no other parameters or else all other parameters have default arguments*

Which of these can be defaulted, and what are the rules for their construction?

Just the  $\&\&$  version, fortunately!





Copyright Jason Turner

@lefticus

[emptycrate.com/go.html](http://emptycrate.com/go.html)

But wait, did you say  
something about `volatile`?



# volatile objects

```
1 struct Data {};  
2  
3 int main() {  
4     volatile Data d;  
5     auto d2 = d; // won't compile  
6 }
```

<https://godbolt.org/z/T5E1cb1da>

volatile objects cannot be copied or moved unless you implement your own appropriate special member.



# volatile objects

```
1 struct Data {  
2     Data(Data &) = default;  
3     Data(const Data &) = default;  
4     Data(Data &&) = default;  
5     // valid, but I have to do these myself  
6     Data(volatile Data &);  
7     Data(volatile Data &&);  
8     Data(const Data &&);  
9     Data(const volatile Data &);  
10    Data(const volatile Data &&);  
11 };
```

<https://godbolt.org/z/c4WKEqabb>

Excluding defaulted parameters, these are the 8 valid copy and move constructors and we can define as many of them as we choose to.





Copyright Jason Turner

@lefticus

[emptycrate.com/go.html](http://emptycrate.com/go.html)

# Assignment Operators?



# Assignment Operators?

Same rules allow for copy/**move assignment** as for copy/move construction. Well... mostly

```
1  struct Data {  
2      Data &operator=(const Data &) = default;  
3      Data &operator=(Data &) = default;  
4      Data &operator=(Data &&) = default;  
5  
6      // valid, but I have to do these myself  
7      Data &operator=(Data); /// new  
8      Data &operator=(volatile Data &);  
9      Data &operator=(volatile Data &&);  
10     Data &operator=(const Data &&);  
11     Data &operator=(const volatile Data &);  
12     Data &operator=(const volatile Data &&);  
13 };
```

<https://godbolt.org/z/TYa9KKTYq>

9 valid forms of copy and **move assignment** operators, 3 can be defaulted (of the expected 2)



# Assignment Operators?

```
1 struct Data { };  
2  
3 int main() {  
4     Data{} = Data{};  
5 }
```

<https://godbolt.org/z/Pej6E6GKW>

Why does this compile?



# Assignment Operators?

```
1 struct Data { };  
2  
3 int main() {  
4     Data{} = Data{};  
5 }
```

<https://godbolt.org/z/Pej6E6GKW>

Why does this compile?

The implicitly defaulted **move assignment** operator is not reference qualified.



# Assignment Operators?

```
1 struct Data {  
2     Data &operator=(Data &&) & = default;  
3 };  
4  
5  
6 int main() {  
7     Data{} = Data{}; // no longer compiles  
8 }
```

<https://godbolt.org/z/KWWTWo9Ej>

No longer compiles because the assignment operator is reference qualified to only work with **lvalue** references, not **rvalue** references.



# Assignment Operators?

```
1 struct Data {  
2     Data &operator=(Data &&) & = default;  
3 };  
4  
5  
6 int main() {  
7     Data{} = Data{}; // no longer compiles  
8 }
```

<https://godbolt.org/z/KWWTWo9Ej>

No longer compiles because the assignment operator is reference qualified to only work with **lvalue** references, not **rvalue** references.

Is this a good change?



# Assignment Operators?

All 15 of the possible things that count as “copy assignment” (default to indicate which can be defaulted)

```
1  struct Data {
2      Data &operator=(Data);
3      Data &operator=(Data) &;
4      Data &operator=(Data) &&;
5
6      Data &operator=(Data &) = default;
7      Data &operator=(Data &) & = default;
8      Data &operator=(Data &) && = default;
9
10     Data &operator=(const Data &) = default;
11     Data &operator=(const Data &) & = default;
12     Data &operator=(const Data &) && = default;
13
14     Data &operator=(volatile Data &);
15     Data &operator=(volatile Data &) &;
16     Data &operator=(volatile Data &) &&;
17
18     Data &operator=(const volatile Data &);
19     Data &operator=(const volatile Data &) &;
20     Data &operator=(const volatile Data &) &&;
21 };
```



Copyright Jason Turner

@lefticus

[emptycrate.com/go.html](http://emptycrate.com/go.html)

# Late Breaking News!



# I Missed a Few!



# Which signatures have been missed?

```
1 struct Data {
2     Data &operator=(Data);
3     Data &operator=(Data) &;
4     Data &operator=(Data) &&;
5
6     Data &operator=(Data &) = default;
7     Data &operator=(Data &) & = default;
8     Data &operator=(Data &) && = default;
9
10    Data &operator=(const Data &) = default;
11    Data &operator=(const Data &) & = default;
12    Data &operator=(const Data &) && = default;
13
14    Data &operator=(volatile Data &);
15    Data &operator=(volatile Data &) &;
16    Data &operator=(volatile Data &) &&;
17
18    Data &operator=(const volatile Data &);
19    Data &operator=(const volatile Data &) &;
20    Data &operator=(const volatile Data &) &&;
21 };
```

<https://godbolt.org/z/bfvxWGnnh>



# Which signatures have been missed?

```
1 struct Data {  
2     Data &operator=(Data);  
3     Data &operator=(Data) &;  
4     Data &operator=(Data) &&;  
5  
6     Data &operator=(Data) const;  
7     Data &operator=(Data) const &;  
8     Data &operator=(Data) const &&;  
9  
10    Data &operator=(Data) volatile;  
11    Data &operator=(Data) volatile &;  
12    Data &operator=(Data) volatile &&;  
13  
14    Data &operator=(Data) const volatile;  
15    Data &operator=(Data) const volatile &;  
16    Data &operator=(Data) const volatile &&;  
17  
18    Data &operator=(Data &) = default;  
19    Data &operator=(Data &) & = default;  
20    Data &operator=(Data &) && = default;  
21  
22    Data &operator=(Data &) const;  
23    Data &operator=(Data &) const &;  
24    Data &operator=(Data &) const &&;
```

<https://godbolt.org/z/bWMjKx968>



# That is all of the unique function signatures.



# But I've still missed a few!

```
1 struct Data {  
2     [constexpr, consteval, '']  
3     retval operator=(Data,  
4         Data&  
5         const Data &  
6         const volatile Data &  
7         volatile Data &)  
8     ['', const, volatile, const volatile]  
9     ['', &, &&]  
10    ['', noexcept]  
11 };
```

<https://godbolt.org/z/zcsbMbeb6>

$(3 * 5 * 4 * 3 * 2) == 360$  possible signatures that the compiler will count as  
“user defined copy assignment operator”



# Assignment Operators?

All 12 of the possible things that count as “move assignment” (default to indicate which can be defaulted)

```
1 struct Data {  
2     Data &operator=(Data &&) = default;  
3     Data &operator=(Data &&) & = default;  
4     Data &operator=(Data &&) && = default;  
5  
6     Data &operator=(const Data &&);  
7     Data &operator=(const Data &&) &;  
8     Data &operator=(const Data &&) &&;  
9  
10    Data &operator=(volatile Data &&);  
11    Data &operator=(volatile Data &&) &;  
12    Data &operator=(volatile Data &&) &&;  
13  
14    Data &operator=(const volatile Data &&);  
15    Data &operator=(const volatile Data &&) &;  
16    Data &operator=(const volatile Data &&) &&;  
17 };
```

<https://godbolt.org/z/PzMzdhnax>



# Assignment Operators?

Plus the `const`, `volatile`, `const volatile` qualified versions.

$(3 * 4 * 4 * 3 * 2) == 288$  possible signatures the compiler will count as  
“user defined move assignment operators”



# Special Members That Can Be Defaulted?



# Special Members That Can Be Defaulted?

Short answer - the ones the compiler is willing to generate for you, and their reference qualified versions.



All in one place for reference, but this is not valid, you cannot have the unqualified and qualified versions of the same member function (C++Weekly Ep 447)

```
1  struct Data {  
2      Data() = default;  
3      Data(Data &) = default;  
4      Data(const Data &) = default;  
5      Data(Data &&) = default;  
6  
7      Data &operator=(Data &&) = default;  
8      Data &operator=(Data &&) & = default;  
9      Data &operator=(Data &&) && = default;  
10  
11     Data &operator=(Data &) = default;  
12     Data &operator=(Data &) & = default;  
13     Data &operator=(Data &) && = default;  
14  
15     Data &operator=(const Data &) = default;  
16     Data &operator=(const Data &) & = default;  
17     Data &operator=(const Data &) && = default;  
18  
19     ~Data() = default;  
20 };
```

<https://godbolt.org/z/9frjMx95a>



However, this is valid:

```
1 struct Data {  
2     Data() = default;  
3     Data(Data &) = default;  
4     Data(const Data &) = default;  
5     Data(Data &&) = default;  
6  
7     Data &operator=(Data &&) & = default;  
8     Data &operator=(Data &&) && = default;  
9  
10    Data &operator=(Data &) & = default;  
11    Data &operator=(Data &) && = default;  
12  
13    Data &operator=(const Data &) & = default;  
14    Data &operator=(const Data &) && = default;  
15  
16    ~Data() = default;  
17 };
```

<https://godbolt.org/z/z1KMz1ebT>

Rule of 11?



However, this is valid:

```
1 struct Data {  
2     Data() = default;  
3     Data(Data &) = default;  
4     Data(const Data &) = default;  
5     Data(Data &&) = default;  
6  
7     Data &operator=(Data &&) & = default;  
8     Data &operator=(Data &&) && = default;  
9  
10    Data &operator=(Data &) & = default;  
11    Data &operator=(Data &) && = default;  
12  
13    Data &operator=(const Data &) & = default;  
14    Data &operator=(const Data &) && = default;  
15  
16    ~Data() = default;  
17 };
```

<https://godbolt.org/z/z1KMz1ebT>

Rule of 11?

No, don't do this!



```
1 struct Data {  
2     constexpr Data() noexcept = default;  
3     constexpr Data(Data &) noexcept = default;  
4     constexpr Data(const Data &) noexcept = default;  
5     constexpr Data(Data &&) noexcept = default;  
6  
7     [[nodiscard]] constexpr Data &operator=(Data &&) & noexcept = default;  
8     Data &operator=(Data &&) && = default;  
9  
10    Data &operator=(Data &) & = default;  
11    Data &operator=(Data &) && = default;  
12  
13    Data &operator=(const Data &) & = default;  
14    Data &operator=(const Data &) && = default;  
15  
16    ~Data() = default;  
17 };
```

<https://godbolt.org/z/qqPG4xoqx>

Just, you know, use the other specifiers and attributes correctly as they make sense.





Copyright Jason Turner

@lefticus

[emptycrate.com/go.html](http://emptycrate.com/go.html)

# What is the rule of 0?



# Rule of 0?

(User defined in this case refers to the programmer mentioning the special member *even just defaulting it!*)

We do nothing at all, get out of the compiler's way, and do what it knows how to do best.

```
1 // Rule of 0. If "Data" has members, their lifetimes are automatically
2 // managed. this is great.
3 struct Data {
4 };
```



# Rule of 3?

This is pre-C++11, but it's important to understand.

```
1 // Rule of 3 - if I have to manage resources in C++98
2 // I need to implement all the special members
3 struct Data {
4     int *ptr;
5     ~Data() { delete ptr; } // now I need the others
6     Data &operator=(const Data &); // do something smart
7     Data(const Data &); // do something smart
8 };
```

<https://godbolt.org/z/b5W4qEEKh>

Why is this insufficient for C++11, and why does it matter?



# Rule of 3?

This is pre-C++11, but it's important to understand.

```
1 // Rule of 3 - if I have to manage resources in C++98
2 // I need to implement all the special members
3 struct Data {
4     int *ptr;
5     ~Data() { delete ptr; } // now I need the others
6     Data &operator=(const Data &); // do something smart
7     Data(const Data &); // do something smart
8 };
```

<https://godbolt.org/z/b5W4qEEKh>

Why is this insufficient for C++11, and why does it matter?

If I define those special members, then the “move” operations are also not defined, so we’ll always implicitly get a copy and never a move.



# Rule of 3?

This is pre-C++11, but it's important to understand.

```
1 // Rule of 3 - if I have to manage resources in C++98
2 // I need to implement all the special members
3 struct Data {
4     int *ptr;
5     ~Data() { delete ptr; } // now I need the others
6     Data &operator=(const Data &); // do something smart
7     Data(const Data &); // do something smart
8 };
```

<https://godbolt.org/z/b5W4qEEKh>

Why is this insufficient for C++11, and why does it matter?

If I define those special members, then the “move” operations are also not defined, so we’ll always implicitly get a copy and never a move.

This can cause considerable performance regressions.



# Rule of 3?

This used to be the default in C++ books and code generators in the C++98 era.

```
1 // .hpp file
2 class Data {
3     Data();
4     ~Data();
5 };
```

<https://godbolt.org/z/v75ohjP1P>

```
1 // .cpp file
2 Data::Data() {}
3 Data::~Data() {}
```

This is terrible and don't even consider it today!



From Howard Hinnant

# Special Members

compiler implicitly declares

| user declares |                     | default constructor | destructor    | copy constructor | copy assignment | move constructor | move assignment |
|---------------|---------------------|---------------------|---------------|------------------|-----------------|------------------|-----------------|
|               | Nothing             | defaulted           | defaulted     | defaulted        | defaulted       | defaulted        | defaulted       |
|               | Any constructor     | not declared        | defaulted     | defaulted        | defaulted       | defaulted        | defaulted       |
|               | default constructor | user declared       | defaulted     | defaulted        | defaulted       | defaulted        | defaulted       |
|               | destructor          | defaulted           | user declared | defaulted        | defaulted       | not declared     | not declared    |
|               | copy constructor    | not declared        | defaulted     | user declared    | defaulted       | not declared     | not declared    |
|               | copy assignment     | defaulted           | defaulted     | defaulted        | user declared   | not declared     | not declared    |
|               | move constructor    | not declared        | defaulted     | deleted          | deleted         | user declared    | not declared    |
|               | move assignment     | defaulted           | defaulted     | deleted          | deleted         | not declared     | user declared   |



# My Version

[https://docs.google.com/spreadsheets/d/1ADQr8J\\_SmhF35YBnwPBg7st0usp=sharing](https://docs.google.com/spreadsheets/d/1ADQr8J_SmhF35YBnwPBg7st0usp=sharing)



# Rule of 4?

Why else is this insufficient?

```
1 struct Data {  
2     int *ptr;  
3     ~Data() { delete ptr; } // now I need the others  
4     Data(const Data &); // do something smart  
5     Data &operator=(const Data &); // do something smart  
6 };
```

<https://godbolt.org/z/Txsr6P8Y9>



# Rule of 4?

Why else is this insufficient?

```
1 struct Data {  
2     int *ptr;  
3     ~Data() { delete ptr; } // now I need the others  
4     Data(const Data &); // do something smart  
5     Data &operator=(const Data &); // do something smart  
6 };
```

<https://godbolt.org/z/Txsr6P8Y9>

The default constructor is also not defined, because I mentioned the copy constructor.



# Rule of 4?

Why else is this insufficient?

```
1 struct Data {  
2     int *ptr = nullptr;  
3     Data() = default;  
4     ~Data() { delete ptr; } // now I need the others  
5     Data(const Data &) = delete;  
6     Data &operator=(const Data &) = delete;  
7 };
```

<https://godbolt.org/z/bhhnPzcW5>

This is now actually safe and sufficient - the type is neither copyable nor movable.



# Rule of 5 / 6

```
1 struct Data {  
2     int *ptr = nullptr;  
3     Data() = default;  
4     ~Data() { delete ptr; } // now I need the others  
5     Data(const Data &) = delete;  
6     Data(Data && other) = delete;  
7     Data &operator=(const Data &) = delete;  
8     Data &operator=(Data &&) = delete;  
9 };
```

<https://godbolt.org/z/8esxssGhn>

IT'S OK FOR THE TYPE TO BE NON-MOVABLE IF YOU DON'T KNOW HOW TO MAKE IT SAFELY MOVABLE!

Do **NOT** just blindly default the operations if you don't understand what they will do (hint: unsafe in this moment).



**IT'S OK FOR THE TYPE TO BE  
NON-MOVABLE IF YOU  
DON'T KNOW HOW TO MAKE  
IT SAFELY MOVABLE!**



Do NOT just blindly `=default`  
the operations if you don't  
understand what they will  
do!



# Alternative Rule of 5 / 6

But wait, what about this:

```
1 | S{} = S{};
```



# Alternative Rule of 5 / 6

But wait, what about this:

```
1 | S{} = S{};
```

Do we really want that work?



# Alternative Rule of 5 / 6

But wait, what about this:

```
1 | s{} = s{};
```

Do we really want that work?

Does

```
1 | int{} = int{};
```

compile?



# Alternative Rule of 5 / 6

But wait, what about this:

```
1 | s{} = s{};
```

Do we really want that work?

Does

```
1 | int{} = int{};
```

compile?

No!



# Alternative Rule of 5 / 6

```
1  #include <utility>
2
3  struct Data {
4      int *ptr = nullptr;
5      Data() = default;
6      Data(const Data &) = delete;
7      Data &operator=(const Data &) = delete;
8
9      ~Data() { delete ptr; }
10
11     Data(Data && other) : ptr{std::exchange(other.ptr, nullptr)}
12     {}
13
14     Data &operator=(Data &&other) & noexcept /// <---
15     {
16         if (&other != this) {
17             delete ptr;
18             ptr = std::exchange(other.ptr, nullptr);
19         }
20     }
21 };
```

<https://godbolt.org/z/oMjxW8z4a>





Copyright Jason Turner

@lefticus

[emptycrate.com/go.html](http://emptycrate.com/go.html)

# Rules of 0, 3, 4, 5, 6



# Rules of 0, 3, 4, 5, 6



# Rules of 0, 3, 4, 5, 6

- Plan to not do anything (best, don't manually manage resources, don't prohibit move operations, keep types trivial): Rule of 0



# Rules of 0, 3, 4, 5, 6

- Plan to not do anything (best, don't manually manage resources, don't prohibit move operations, keep types trivial): Rule of 0
- If you want the type to not be copyable, delete the copy operations only: Rule of 3 (but don't actually do this!)



# Rules of 0, 3, 4, 5, 6

- Plan to not do anything (best, don't manually manage resources, don't prohibit move operations, keep types trivial): Rule of 0
- If you want the type to not be copyable, delete the copy operations only: Rule of 3 (but don't actually do this!)
- You probably need to default the default constructor at some point: Rule of 4



# Rules of 0, 3, 4, 5, 6

- Plan to not do anything (best, don't manually manage resources, don't prohibit move operations, keep types trivial): Rule of 0
- If you want the type to not be copyable, delete the copy operations only: Rule of 3 (but don't actually do this!)
- You probably need to default the default constructor at some point: Rule of 4
- If you mention any special member function, deal with all of them: Rule of 5



# Rules of 0, 3, 4, 5, 6

- Plan to not do anything (best, don't manually manage resources, don't prohibit move operations, keep types trivial): Rule of 0
- If you want the type to not be copyable, delete the copy operations only: Rule of 3 (but don't actually do this!)
- You probably need to default the default constructor at some point: Rule of 4
- If you mention any special member function, deal with all of them: Rule of 5
- Remember: if you are enabling assignment, there's a good chance you only want the l-value reference qualified assignment operators.



# Rules of 0, 3, 4, 5, 6

- Plan to not do anything (best, don't manually manage resources, don't prohibit move operations, keep types trivial): Rule of 0
- If you want the type to not be copyable, delete the copy operations only: Rule of 3 (but don't actually do this!)
- You probably need to default the default constructor at some point: Rule of 4
- If you mention any special member function, deal with all of them: Rule of 5
- Remember: if you are enabling assignment, there's a good chance you only want the l-value reference qualified assignment operators.
- And there's a good chance again that you'll need the default constructor: Rule of 6



# Rules of 0, 3, 4, 5, 6

Two of these are risky...



# Rules of 0, ~~3~~, ~~4~~, 5, 6

- Plan to not do anything (best, don't manually manage resources, don't prohibit move operations, keep types trivial): Rule of 0
- ~~If you want the type to not be copyable, delete the copy operations only: Rule of 3 (but don't actually do this!)~~
- ~~You probably need to default the default constructor at some point: Rule of 4~~
- If you mention any special member function, deal with all of them: Rule of 5
- Remember: if you are enabling assignment, there's a good chance you only want the l-value reference qualified assignment operators.
- And there's a good chance again that you'll need the default constructor: Rule of 6



# Rules of 0, 5, 6

- Plan to not do anything (best, don't manually manage resources, don't prohibit move operations, keep types trivial): Rule of 0
- If you mention any special member function, deal with all of them: Rule of 5
- Remember: if you are enabling assignment, there's a good chance you only want the l-value reference qualified assignment operators.
- And there's a good chance again that you'll need the default constructor: Rule of 6



# Do you really want a default constructor tho?



# Do you really want a default constructor tho?

Let's see:



# Do you really want a default constructor tho?

Let's see:

- It lets me resize a container if I have one



# Do you really want a default constructor tho?

Let's see:

- It lets me resize a container if I have one
- But it also lets users do things like this:



# Do you really want a default constructor tho?

Let's see:

- It lets me resize a container if I have one
- But it also lets users do things like this:

```
1  #include <memory>
2
3  int main() {
4      std::unique_ptr<int> ptr;
5      ptr = std::make_unique<int>(42); /// but why??
6  }
```

<https://godbolt.org/z/1K7a7vzTv>



# Rules of 0, 5, 6

- Plan to not do anything (best, don't manually manage resources, don't prohibit move operations, keep types trivial): Rule of 0
- If you mention any special member function, deal with all of them: Rule of 5
- Remember: if you are enabling assignment, there's a good chance you only want the l-value reference qualified assignment operators.
- ~~And there's a good chance again that you'll need the default constructor: Rule of 6~~
- Do something with the default constructor (`=default`, `=delete`, but don't define, use in-class initializers instead)



# Rules of 0, 6

- Plan to not do anything (best, don't manually manage resources, don't prohibit move operations, keep types trivial): Rule of 0
- If you mention any special member function, deal with all of them, including the default constructor: Rule of 6
- Remember: if you are enabling assignment, there's a good chance you only want the l-value reference qualified assignment operators.



# Using In-Class Initializers (Rule of 6)

```
1 // prefer this
2 struct unique_ptr {
3     int *ptr = nullptr; /// good
4     unique_ptr() = default;
5 }; https://godbolt.org/z/hMhKoh57G
```

```
1 // not this
2 struct unique_ptr_bad {
3     int *ptr; /// bad
4     unique_ptr_bad() : ptr{nullptr} {}
5 }; https://godbolt.org/z/c8ohvTjK6
```



How are we looking for  
time? There's still a lot to  
go!



# What other functions can the compiler generate for us?



# What other functions can the compiler generate for us?

Comparison operations (C++20)!



# What other functions can the compiler generate for us?

Comparison operations (C++20)!

Which comparisons can it generate for us?



# What other functions can the compiler generate for us?

Comparison operations (C++20)!

Which comparisons can it generate for us?

All of them! (potentially)



# Defaulted comparison operators

```
1 #include <compare> /// necessary for <=>
2
3 struct S {
4     auto operator<=>(const S &) const & = default;
5     bool operator==(const S &) const = default;
6     bool operator!=(const S &) const = default;
7     bool operator<(const S &) const = default;
8     bool operator>(const S &) const = default;
9     bool operator<=(const S &) const = default;
10    bool operator>=(const S &) const = default;
11 };
```

<https://godbolt.org/z/713nG6bhj>

This is completely valid, and pointless! (We'll get into this in a moment.)



```
1  #include <compare>
2
3  struct S {
4      auto operator<=>(const S &) const & = default;
5      bool operator==(const S &) const = default;
6      bool operator!=(const S &) const = default;
7      bool operator<(const S &) const = delete;
8      bool operator>(const S &) const = default;
9      bool operator<=(const S &) const = delete;
10     bool operator>=(const S &) const = default;
11 };
```

<https://godbolt.org/z/68oWYee74>

This is also valid...



```
1  #include <compare>
2
3  struct S {
4      auto operator<=>(const S &) const & = default;
5      bool operator==(const S &) const = default;
6      bool operator!=(const S &) const = default;
7      bool operator<(const S &) const = delete;
8      bool operator>(const S &) const = default;
9      bool operator<=(const S &) const = delete;
10     bool operator>=(const S &) const = default;
11 };
```

<https://godbolt.org/z/68oWYee74>

This is also valid...

hmmm...



# Defaulted comparison operators

```
1  #include <compare>
2
3  struct Rational {
4      int numerator;
5      int denominator;
6      /// gives us all of the comparison operators
7      auto operator<=>(const Rational &) const & = default;
8  };
```

<https://godbolt.org/z/cEWsq59nb>

Is this a reasonable thing to do?



# Defaulted comparison operators

```
1  #include <compare>
2
3  struct Rational {
4      int numerator;
5      int denominator;
6      /// gives us all of the comparison operators
7      auto operator<=>(const Rational &) const & = default;
8  };
```

<https://godbolt.org/z/cEWsq59nb>

Is this a reasonable thing to do?

Probably not, the ordering makes no sense (lexicographical ordering of the members by order of declaration).

A smaller numerator always compares as less than.

So in this world:  $1/1 < 1/10$ .



# Defaulted comparison operators

```
1 struct Rational {  
2     int numerator;  
3     int denominator;  
4     auto operator==(const Rational &) const -> bool = default;  
5 };
```

<https://godbolt.org/z/xcnE7xMGs>

Is this a reasonable thing to do?



# Defaulted comparison operators

```
1 struct Rational {  
2     int numerator;  
3     int denominator;  
4     auto operator==(const Rational &) const -> bool = default;  
5 };
```

<https://godbolt.org/z/xcnE7xMGs>

Is this a reasonable thing to do?

Probably, yes, probably almost always.



# Defaulted comparison operators

```
1 struct Rational {  
2     int numerator;  
3     int denominator;  
4     auto operator==(const Rational &) const -> bool = default;  
5 };
```

<https://godbolt.org/z/xcnE7xMGs>

Is this a reasonable thing to do?

Probably, yes, probably almost always.

What about in this particular case?



# Defaulted comparison operators

```
1 struct Rational {  
2     int numerator;  
3     int denominator;  
4     auto operator==(const Rational &) const -> bool = default;  
5 };
```

<https://godbolt.org/z/xcnE7xMGs>

Is this a reasonable thing to do?

Probably, yes, probably almost always.

What about in this particular case?

Maybe?



# Defaulted comparison operators

```
1 struct Rational {  
2     int numerator;  
3     int denominator;  
4     auto operator==(const Rational &) const -> bool = default;  
5     auto operator!=(const Rational &) const -> bool = default;  
6 };
```

<https://godbolt.org/z/vqbzdnMjK>

Is this a reasonable thing to do?



# Defaulted comparison operators

```
1 struct Rational {  
2     int numerator;  
3     int denominator;  
4     auto operator==(const Rational &) const -> bool = default;  
5     auto operator!=(const Rational &) const -> bool = default;  
6 };
```

<https://godbolt.org/z/vqbzdnMjK>

Is this a reasonable thing to do?

No - if I have `==`, I get `!=` for free!



# Defaulted comparison operators

```
1  #include <compare>
2  #include <numeric>
3
4  struct Rational {
5      int numerator;
6      int denominator;
7
8      [[nodiscard]] constexpr auto operator<=>(const Rational &rhs) const noexcept -> std::strong_ordering {
9          const auto lcm = std::lcm(denominator, rhs.denominator);
10         const auto lhs_val = numerator * (lcm / denominator);
11         const auto rhs_val = rhs.numerator * (lcm / rhs.denominator);
12         return lhs_val <=> rhs_val;
13     }
14 };
15
16 int main() {
17     constexpr auto lhs = Rational{4,9};
18     constexpr auto rhs = Rational{3,6};
19     static_assert(lhs < rhs);
20 }
```

<https://godbolt.org/z/58ojf99W4>



# Defaulted comparison operators

```
1  #include <compare>
2  #include <numeric>
3
4  struct Rational {
5      int numerator;
6      int denominator;
7
8      [[nodiscard]] constexpr auto operator<=>(const Rational &rhs) const noexcept -> std::strong_ordering {
9          const auto lcm = std::lcm(denominator, rhs.denominator);
10         const auto lhs_val = numerator * (lcm / denominator);
11         const auto rhs_val = rhs.numerator * (lcm / rhs.denominator);
12         return lhs_val <=> rhs_val;
13     }
14 };
15
16 int main() {
17     constexpr auto lhs = Rational{4,8};
18     constexpr auto rhs = Rational{3,6};
19     static_assert(lhs == rhs);
20 }
```

<https://godbolt.org/z/8erEWWY97>



# Defaulted comparison operators

```
1  #include <compare>
2  #include <numeric>
3
4  struct Rational {
5      int numerator;
6      int denominator;
7
8      [[nodiscard]] constexpr auto operator<=>(const Rational &rhs) const noexcept -> std::strong_ordering {
9          const auto lcm = std::lcm(denominator, rhs.denominator);
10         const auto lhs_val = numerator * (lcm / denominator);
11         const auto rhs_val = rhs.numerator * (lcm / rhs.denominator);
12         return lhs_val <=> rhs_val;
13     }
14 };
15
16 int main() {
17     constexpr auto lhs = Rational{4,8};
18     constexpr auto rhs = Rational{3,6};
19     static_assert(lhs == rhs);
20 }
```

<https://godbolt.org/z/8erEWWY97>

Error: equality operator not defined :(



# Defaulted comparison operators

If you give a user provided three-way spaceship operator, you don't get `==` (there is actually a good reason for this, if we have time...)



# Defaulted comparison operators

```
1  #include <compare>
2  #include <numeric>
3
4  struct Rational {
5      int numerator;
6      int denominator;
7
8      [[nodiscard]] constexpr auto operator<=>(const Rational &rhs) const noexcept -> std::strong_ordering {
9          const auto lcm = std::lcm(denominator, rhs.denominator);
10         const auto lhs_val = numerator * (lcm / denominator);
11         const auto rhs_val = rhs.numerator * (lcm / rhs.denominator);
12         return lhs_val <=> rhs_val;
13     }
14     auto operator==(const Rational &) const = default;
15 };
16
17 int main() {
18     constexpr auto lhs = Rational{4,8};
19     constexpr auto rhs = Rational{3,6};
20     static_assert(lhs == rhs);
21 }
```

<https://godbolt.org/z/19jq98bez>



# Defaulted comparison operators

```
1  #include <compare>
2  #include <numeric>
3
4  struct Rational {
5      int numerator;
6      int denominator;
7
8      [[nodiscard]] constexpr auto operator<=>(const Rational &rhs) const noexcept -> std::strong_ordering {
9          const auto lcm = std::lcm(denominator, rhs.denominator);
10         const auto lhs_val = numerator * (lcm / denominator);
11         const auto rhs_val = rhs.numerator * (lcm / rhs.denominator);
12         return lhs_val <=> rhs_val;
13     }
14     auto operator==(const Rational &) const = default;
15 };
16
17 int main() {
18     constexpr auto lhs = Rational{4,8};
19     constexpr auto rhs = Rational{3,6};
20     static_assert(lhs == rhs);
21 }
```

<https://godbolt.org/z/19jq98bez>

 Error: fails to compile because the explicitly defaulted `operator==` does not use my user defined `operator<=>`

# Defaulted comparison operators

```
1  #include <compare>
2  #include <numeric>
3
4  struct Rational {
5      int numerator;
6      int denominator;
7
8      [[nodiscard]] constexpr auto operator<=>(const Rational &rhs) const noexcept -> std::strong_ordering {
9          const auto lcm = std::lcm(denominator, rhs.denominator);
10         const auto lhs_val = numerator * (lcm / denominator);
11         const auto rhs_val = rhs.numerator * (lcm / rhs.denominator);
12         return lhs_val <=> rhs_val;
13     }
14     [[nodiscard]] constexpr auto operator==(const Rational &rhs) const noexcept -> bool {
15         return (*this <=> rhs) == std::weak_ordering::equivalent;
16     }
17 };
18
19 int main() {
20     constexpr auto lhs = Rational{4,8};
21     constexpr auto rhs = Rational{3,6};
22     static_assert(lhs == rhs);
23 }
```

<https://godbolt.org/z/5WojjGE71>



# Defaulted comparison operators

```
1  #include <compare>
2  #include <numeric>
3
4  struct Rational {
5      int numerator;
6      int denominator;
7
8      [[nodiscard]] constexpr auto operator<=>(const Rational &rhs) const noexcept -> std::strong_ordering {
9          const auto lcm = std::lcm(denominator, rhs.denominator);
10         const auto lhs_val = numerator * (lcm / denominator);
11         const auto rhs_val = rhs.numerator * (lcm / rhs.denominator);
12         return lhs_val <=> rhs_val;
13     }
14     [[nodiscard]] constexpr auto operator==(const Rational &rhs) const noexcept -> bool {
15         return (*this <=> rhs) == std::weak_ordering::equivalent;
16     }
17 };
18
19 int main() {
20     constexpr auto lhs = Rational{4,8};
21     constexpr auto rhs = Rational{3,6};
22     static_assert(lhs == rhs);
23 }
```

<https://godbolt.org/z/5WojjGE71>



Yay!

# Comparison operators rule of 1, 2

- Almost always explicitly default `operator==` (rule of 1)
- If you need to provide your own `<=>` implementation, you *must* also provide your own `operator==`. It can be provided in terms of your `<=>` if that makes sense. (rule of 2)



# Comparison operators rule of N?

```
1 #include <compare> /// necessary for <=>
2
3 struct Float {
4     auto operator<=>(const Float &) const & = default;
5     bool operator==(const Float &) const = delete;
6     bool operator!=(const Float &) const = delete;
7
8     float value;
9 };
```

<https://godbolt.org/z/ne6Gs11sc>

Are there types where it might be desirable to *remove* some of the comparison operations to make the type more correct?



# Comparison operators rule of N?

```
1 #include <compare> /// necessary for <=>
2
3 void use(auto);
4
5 struct Float {
6     auto operator<=>(const Float &) const & = default;
7     bool operator==(const Float &) const = delete;
8     bool operator!=(const Float &) const = delete;
9
10    // I'm lifting < from "usable" to "defined"
11    bool operator<(const Float &) const = default;
12
13    float value;
14 };
15
16 int main() {
17     use(&Float::operator<); // compiles
18     use(&Float::operator>); // does not compile
19 }
```

<https://godbolt.org/z/4njan8fw1>



# Comparison operators rule of N?

```
1 #include <compare> /// necessary for <=>
2
3 void use(auto);
4
5 struct Float {
6     auto operator<=>(const Float &) const & = default;
7     bool operator==(const Float &) const = delete;
8     bool operator!=(const Float &) const = delete;
9
10    // I'm lifting < from "usable" to "defined"
11    bool operator<(const Float &) const = default;
12
13    float value;
14 };
15
16 int main() {
17     use(&Float::operator<); // compiles
18     use(&Float::operator>); // does not compile
19 }
```

<https://godbolt.org/z/4njan8fw1>

I'm honestly not sure what to do with this knowledge.





Copyright Jason Turner

@lefticus

[emptycrate.com/go.html](http://emptycrate.com/go.html)

# Anything Else That Can Be Defaulted?



# C++20's `operator++(int)`



# C++29's `operator++(int)`

first - what is this?



# C++29's `operator++(int)`

first - what is this?

It's the post-increment operator.

```
1  struct Iterator {  
2      int *ptr;  
3      // pre  
4      constexpr auto operator++() noexcept -> Iterator & {  
5          ++ptr;  
6          return *this;  
7      }  
8  
9      // post  
10     // no reason to call the post increment operator  
11     // if you plan to throw away the results  
12     [[nodiscard]] constexpr auto operator++(int) noexcept -> Iterator {  
13         auto prev = *this;  
14         ++(*this);  
15         return prev;  
16     }  
17 };
```

<https://godbolt.org/z/xe9EcasfW>



# C++29's `operator++(int)=default;`

```
1  struct Iterator {  
2      int *ptr;  
3      // pre  
4      constexpr auto operator++() noexcept -> Iterator & {  
5          ++ptr;  
6          return *this;  
7      }  
8  
9      // post  
10     [[nodiscard]] constexpr auto operator++(int) noexcept -> Iterator = default;  
11 };
```

<https://godbolt.org/z/bfMT4djGb>



# C++29's `operator++(int)`



# C++29's `operator++(int)`

- Rule of 0: Do nothing if it doesn't make sense



# C++29's `operator++(int)`

- Rule of 0: Do nothing if it doesn't make sense
- Rule of 1: If you have a prefix increment operator, `=default` or `=delete` the postfix increment.



# Rules of...

- Special Members: (0, 6)
- Comparison Operators: (1, 2, N)
- `operator++`: (0, 1)
- `operator--`: (0, 1)

Which gives us the rules of:

1, 2, 3, 7, 8, 8, 8, 9, 9, 10, N?



# Complete-ish example

<https://compiler-explorer.com/z/sd7snM1xc>



```

1  #include <memory>
2
3  template <typename ValueType>
4  class ring_buffer {
5      std::size_t m_size;
6      std::size_t m_index{0};
7      std::unique_ptr<ValueType[]> m_data;
8
9  public:
10     using value_type = ValueType;
11
12     [[nodiscard]] constexpr auto begin(this auto&& self) -> auto* {
13         return &self.m_data[0];
14     }
15
16     [[nodiscard]] constexpr auto end(this auto&& self) -> auto* {
17         return &self.m_data[self.m_size];
18     }
19
20     template<typename Self>
21     [[nodiscard]] constexpr auto operator*(this Self && self) -> auto && {
22         return std::forward_like<Self>(self.m_data[self.m_index]);
23     }
24

```

<https://godbolt.org/z/3W13E9oza>





Copyright Jason Turner

@lefticus

[emptycrate.com/go.html](http://emptycrate.com/go.html)

# p2953



# p2953

<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/p2953r5.html>

*Current C++ permits explicitly-defaulted special members to differ from their implicitly-defaulted counterparts in various ways, including parameter type and ref-qualification. This permits implausible declarations like `A& operator=(const A&) && = default`, where the left-hand operand is `rvalue`-ref-qualified. We propose to forbid such declarations.*



# p2953

Seems this was accepted for C++29. The wording of “implausible declarations” is very odd to me - since the non-qualified versions provide both operations anyhow...



# TL;DW - Special members (C++11)

- Prefer “rule of 0”
- Do intelligent things with all 6 special members if you manage resources
- Don’t permit the `&&` qualified assignment operator (only provide the `&` one)



# TL;DW - Comparisons (C++20)

- Almost always default either the `<=>` or the `==`
- If you need to provide your own `<=>`, then also provide your own `==`
- (don't bother with the others unless you have some reason to `delete` one of them or `default` one to remove it or 'lift' it)



# TL;DW - postfix operators (C++29)

- If you define prefix increment or prefix decrement, default the postfix versions (and make them `[[nodiscard]]`!)



# TL;DW - All operators

Consider making any explicitly defined or defaulted operator

`[[nodiscard]]`, `constexpr` and possibly `noexcept` depending on context.



# Rules of 0,1,2,3,4,5,6,7,8,9,10?



# Rules of N.



**Rule: Be intentional with  
special members,  
comparisons, and prefix  
operators.**



# Jason Turner

## C++ Weekly

- Weekly videos since March, 2016
- 125k+ subscribers, 520+ weeks straight

<https://www.youtube.com/@cppweekly>



# Jason Turner

- Author
  - C++ Best Practices, C++23 Best Practices
  - OpCode, Copy and Reference, Object Lifetime Puzzlers
  - Programming Puzzles Big Book
  - <https://amzn.to/3xWh8Ox>
  - [https://leanpub.com/u/jason\\_turner](https://leanpub.com/u/jason_turner)



# Jason Turner

- Developer
  - <https://cppbestpractices.com>
  - <https://github.com/lefticus>
  - <https://github.com/cpp-best-practices>
- Microsoft MVP for C++, 2015-present



# Jason Turner - Training

<https://emptycrate.com/training.html>

How to get my training:

1. Have me come to your company on-site for dynamic customized training where you already are - generally the most economical option for groups (CA, DE, NL, RO, CZ, JP, US, PL, SE, ...)
2. Come to a conference workshop
  - ACCU On Sea (Folkestone, UK, ~June)
  - CppCon (Aurora, CO, US, ~Sept)
  - NDC TechTown (Kongsberg, NO, ~Sept)
  - C++ Under the Sea (Breda, NL, ~Oct)



And possibly others

# My Motivations



# My Motivations

- The world runs on software. Basically everything does.



# My Motivations

- The world runs on software. Basically everything does.
- Have you ever had to reboot your toaster? (I have!)



# My Motivations

- The world runs on software. Basically everything does.
- Have you ever had to reboot your toaster? (I have!)
- I want to live in a world with better, safer software, cars, spaceships, toasters...



# My Motivations

- The world runs on software. Basically everything does.
- Have you ever had to reboot your toaster? (I have!)
- I want to live in a world with better, safer software, cars, spaceships, toasters...
- This is why I've given away so much material and continue to do so.



# My Motivations

- The world runs on software. Basically everything does.
- Have you ever had to reboot your toaster? (I have!)
- I want to live in a world with better, safer software, cars, spaceships, toasters...
- This is why I've given away so much material and continue to do so.
- This is also why I focus on Best Practices in my training material.



# My Motivations

- The world runs on software. Basically everything does.
- Have you ever had to reboot your toaster? (I have!)
- I want to live in a world with better, safer software, cars, spaceships, toasters...
- This is why I've given away so much material and continue to do so.
- This is also why I focus on Best Practices in my training material.
- The work you are doing matters. It affects peoples lives!



# Jason Turner

<https://emptycrate.com/go.html>

